

Nystrom 모드 코드 실행 경로 & 수식 walkthrough

대상 스크립트: [vinfo/n05_1.sh](#) · 예시 한 줄 기준으로 코드를 타고 가며 설명합니다. (데이터셋 sst2, 모델 bert, n_class=2)

```
python task_shapley.py          --seed 2024 --dataset_name sst2 \
    --num_train_dp 5000 --val_sample_num 872 \
    --approximate nystrom --nystrom_d 5 --nystrom_lambda_ 1e-2 \
    --inv_lambda_ 1e-6 --tmc_iter 500
python task_data_selection.py (... same args ...)
```

기호: $n = \text{num_train_dp} = 5000$, $t = \text{val_sample_num} = 872$, $d = 250$ (= nystrom_d 5% x n), s = TMC 한 스텝에서 선택된 부분집합 크기(1.. n), $C=2$ = 클래스 수.

파이프라인은 3단계입니다: ① `task_ntk.py` 가 NTK 행렬을 1회 계산/캐시 → ② `task_shapley.py` 가 Nystrom 근사 위에서 TMC - Shapley로 데이터 가치(랭킹)를 계산 → ③ `task_data_selection.py` 가 그 랭킹으로 데이터를 선택해 정확도를 평가.

단계 1. task_ntk.py — eNTK 행렬 K 계산 및 캐시

NTK(정확히는 empirical NTK)는 모든 단계가 공유하는 입력입니다. 이 단계는 한 번만 돌면 됩니다.

1-1) 데이터 샘플링

`task_ntk.py:85` 의 `load_dataset("sst2")` 로 sst2를 불러오고, `task_ntk.py:108` 의 `train.shuffle(seed).select(range(5000))` 로 train 5000개를 고정 시드(2024)로 샘플 → `sampled_idx`. `task_ntk.py:128` 에서 validation 872개를 `np.random.choice` 로 샘플 → `sampled_val_idx`.

1-2) NTK 계산

`task_ntk.py:177` 이 `probe_model.compute_ntk(train_set, val_set)` (`probe.py:327`) 를 호출해 eNTK를 계산합니다. 결과 텐서 모양은 (클래스/커널 축 1) x (train+val) x (train) 입니다:

```
K.shape = [1, n + t, n] = [1, 5872, 5000]
K[0, :n, :] = K(X_train, X_train) # train-train, n x n
K[0, n:, :] = K(X_val, X_train) # val-train, t x n
```

코드 위치 — `probe.py:327 compute_ntk` → `compute_ntk(self.model, train_set, val_set, ...)`. 수치 overflow 방지를 위해 평균이 크면 `K/10000` 정규화 (`probe.py:348`, 로드 시엔 `probe.py:362`).

1-3) 캐시 저장

`task_ntk.py:196` 에서 `{ntk, sampled_idx, sampled_val_idx, meta}` 를 pickle로 저장:

```
freeshap_res/ntk/sst2/bert_seed2024_num5000_val872_signFalse.pkl
```

단계 2. task_shapley.py — Nystrom 위에서 TMC-Shapley

2-1) 인자 파싱 & nystrom_d(%) → 실제 landmark 수 d 변환

`--nystrom_d` 는 `eigen_rank`와 동일하게 "퍼센트"로 해석됩니다 (`task_shapley.py:81`, `type=float`). 실제 변환은 `task_shapley.py:120-121`:

```
nystrom_d_pct = args.nystrom_d # = 5.0
d = int(num_train_dp * nystrom_d_pct/100) # = int(5000*5/100) = 250
```

즉 이 실행에서 landmark는 $d = 250$ 개입니다 (eigen rank $r=250$ 과 같은 차원으로 맞춘 비교).

2-2) 모드/파라미터 설정

task_shapley.py:154 → probe.approximate("nystrom") (probe.py:273, self.approximate_ntk="nystrom" 설정). 이어서 task_shapley.py:160 → set_nystrom_params(d=250, lam=1e-2, landmark_seed=seed=2024, jitter=1e-8) (probe.py:299).

주의 — landmark_seed=seed 로 전달되어 landmark 샘플링도 실험 시드(2024)를 사용합니다. 별도 landmark seed는 없습니다.

2-3) NTK 주입 & train labels

task_shapley.py:204 에서 단계 1의.pkl을 로드 → get_cached_ntk(ntk) (probe.py:357), get_train_labels(train_set) (probe.py:368, 길이 5000의 라벨 벡터).

2-4) [핵심] Nystrom feature 전처리 — TMC 진입 전 1회

task_shapley.py:247 → prepare_nystrom_regression() (probe.py:421) 가 NystromNTKRegression(rank=250, lam=1e-2, landmark_seed=2024, jitter=1e-8) (probe.py:435) 를 생성합니다. 실제 수식은 entks/ntk_regression.py:445 _precompute_eigen_features (Nystrom override)에서 계산됩니다:

```
# entks/ntk_regression.py:461-476
d = min(rank, n) = 250 # num landmarks
S_land = RandomState(2024).choice(n, 250, replace=False) # :464
W = K_sym[S_land, S_land] # (d x d) = (250 x 250) :468
C_tr= K_sym[:, S_land] # (n x d) = (5000 x 250) :469
C_te= K(val, :)[:, S_land] # (t x d) = (872 x 250) :470
L_w = cholesky(W + 1e-8 * I) # (d x d) :473
Phi_tr = C_tr @ L_w^{-T} # (n x d) = (5000 x 250) :475
Phi_te = C_te @ L_w^{-T} # (t x d) = (872 x 250) :476
```

수학적 의미: $\Phi_{tr} @ \Phi_{tr}^T = C_{tr} W^{-1} C_{tr}^T$ 는 K의 Nystrom 근사(rank-d). TMC가 사용하는 것은 이 Φ_{tr} (5000x250), Φ_{te} (872x250) 뿐이며, $W \cdot C \cdot K_{sym}$ 은 여기서 버려집니다. eigen 모드는 같은 자리에서 K의 top-250 고유분해로 Phi를 만든다는 점만 다릅니다(같은 모양 5000x250 / 872x250).

2-5) TMC-Shapley 루프

task_shapley.py:313 → dshap_com.run(..., iteration=500, per_point=True, early_stopping=True) (dvutils/Data_Shapley.py:338). 먼저 get_full_score() (Data_Shapley.py:409) 로 전체 점수 기준선을 잡고, Data_Shapley.py:412 의 for curr_iter in range(500) 가 tmc_one_iteration_per_point (Data_Shapley.py:205) 를 500번 호출합니다. 한 iteration:

```
idxs = np.random.permutation(5000) # random permutation :214
selected = []
for n, idx in enumerate(idxs): # add one point
    selected.append(idx)
_, new_acc = kernel_regression(selected, val, per_point=True) :230
marginal_contribs[idx] = new_score - old_score :234
# early stopping: stop when near full_score (2x) :236-244
sv_result[idx] += (1/500) * marginal_contribs[idx] :212
```

즉 각 점의 Shapley 값 = "무작위 순열에서 그 점을 추가했을 때 val 정확도 한계기여"를 500개 순열로 평균한 값. early stopping은 부분집합 점수가 full score의 tolerance(0.05) 이내로 수렴하면 그 순열을 조기 종료합니다 (Data_Shapley.py:239).

2-6) TMC 한 스텝의 실제 예측 = Nystrom ridge

`Data_Shapley.py:230` 의 `kernel_regression` 는 `nystrom` 모드에서 `probe.py:513-515` 분기를 타고 `self.nystrom_regression(train_indices)` 를 호출합니다. 이 forward는 `NystromNTKRegression` 가 `EigenNTKRegression.forward` 를 상속하므로 `entks/ntk_regression.py:301` 와 동일합니다. 선택 부분집합 크기 `s`에 따라 두 갈래:

```
PhiS = Phi_tr[selected] # (s x d) :321
# (A) s >= d : feature space ridge (d x d inverse) :355
B = PhiS^T @ PhiS # (d x d) = (250 x 250)
W_hat = (B + lam*I)^{-1} @ PhiS^T @ Y_S # ridge, lam=1e-2
pred = Phi_te @ W_hat # (t x C) = (872 x 2)
# s>=500: Sherman-Morrison rank-1 update (speedup) :358-375
# (B) s < d : kernel space (s x s inverse) :332
K_SS = PhiS @ PhiS^T # (s x s)
alpha = (K_SS + lam*I)^{-1} @ Y_S
pred = Phi_te @ PhiS^T @ alpha
```

여기서 `lam = nystrom_lambda(1e-2)`가 ridge 정규화로 들어갑니다. eigen 모드도 forward 코드가 똑같고(상속) `d`만 같으면 계산량이 동일합니다.

2-7) 결과 저장

`run()` 의 반환 `dv_result` 모양은 `[n, num_metric, t] = [5000, 2, 872]` (`Data_Shapley.py:363`; metric 0=음의 entropy, 1=accuracy). `task_shapley.py:335` 에서 pkl로 저장:

```
freeshap_res/shapley/sst2/nystrom/
bert_seed2024_num5000_val872_nys5.0_nyslam1e-02_invlam1e-06
_cholesky_float32_signFalse_earlystopTrue_tmc500.pkl
```

단계 3. task_data_selection.py — 랭킹 기반 데이터 선택 평가

단계 2의 Shapley pkl을 읽어(`task_data_selection.py:159`), 데이터 가치 랭킹을 만들고 선택 비율별로 정확도를 잹니다.

3-1) Shapley 랭킹 생성 (1회)

```
dv_result # [5000, 2, 872] (from stage 2)
acc_contrib = dv_result[:, 1, :] # accuracy contrib :208
acc_sum = acc_contrib.sum(axis=1) # sum over val :209
sorted_idx = argsort(acc_sum)[::-1] # descending rank :211
```

3-2) 두 가지 예측기로 평가 (dual-mode)

같은 랭킹의 top-k에 대해 정확도를 두 방식으로 측정합니다:

① NYSTROM mode (`task_data_selection.py:280-299`): `approximate_ntk="nystrom"` 그대로 → `nystrom_regression` (rank-250 근사)로 예측.

② INV mode (`task_data_selection.py:317-336`): `approximate_ntk="inv"` 로 전환(:318) + `set_inv_params(lam=1e-6)` 후 같은 top-k를 정확한 full eNTK 역행렬로 예측. 이때 `shapleyNTKRegression` (`ntk_regression.py:489`) 가 쓰여:

```
# inv prediction (exact kernel), probe.py:535-552 / ntk_regression.py:520
K_SS = K[selected, selected] # (k x k) original kernel
beta = (K_SS + 1e-6*I)^{-1} @ y_onehot
pred = K(val, selected) @ beta # (t x C)
```

따라서 INV 블록은 "Nystrom - TMC가 만든 랭킹"을 "정확한 모델"로 평가한 것 — 랭킹 품질의 기준점이 됩니다.

3-3) 결과 파일

`task_data_selection.py:386` 이 선택 비율 1~100%별 top/random 정확도(INV · Nystrom 둘 다)를 기록:

```
freeshap_res/data_selection/sst2/nystrom/predictions/
<shapley filename above>_predictions.txt
```

부록 A. TMC 진입 전 보유 행렬 & 모양 (eigen vs nystrom)

항목	eigen	nystrom
분해 대상	$K_{sym} (n \times n) = 5000 \times 5000$	$W = K(S, S) (d \times d) = 250 \times 250$
분해 방법	top - d 고유분해	W의 Cholesky (full)
남기는 Φ_{tr}	$n \times d = 5000 \times 250$	$n \times d = 5000 \times 250$
남기는 Φ_{te}	$t \times d = 872 \times 250$	$t \times d = 872 \times 250$
전처리 비용(1회)	$O(n^3)$ or $O(n^2 d)$	$O(n d^2 + d^3)$
실측 (timing.pkl)	eigendecomp 7.26s	feature 0.14s

→ 전처리 후 두 방식 모두 같은 모양의 Φ 두 개로 귀결. 차이는 1회 전처리 비용과 Φ 의 수치 내용뿐.

부록 B. TMC iteration 계산복잡도 (부분집합 s, 한 번 예측)

forward 코드가 상속으로 동일하고 d도 같으므로 eigen=nystrom으로 일치합니다.

구간	연산	복잡도 (eigen = nystrom)
$s < d$ (kernel)	$K_{SS} = \Phi S \Phi^T S + \text{Cholesky}$	$O(s^2 d + s^3)$
$s \geq d$ (feature)	$B = \Phi S^T \Phi S (d \times d) + \text{ridge solve}$	$O(s d^2 + d^3)$
$s \geq 500$ (incr.)	Sherman - Morrison rank - 1 update	$O(d^2)$
prediction	$\Phi_{te} @ W_{hat}$	$O(t d)$

→ 모든 항이 s, d에만 의존. eigen rank r = nystrom d로 맞추면 TMC 비용 전부 동일.

부록 C. 핵심 파일 한눈에

파일	역할	핵심 함수 / 라인
task_ntk.py	eNTK 계산 · 캐시	compute_ntk → :177
task_shapley.py	Nystrom TMC - Shapley 실행	:120 d-conv, :247 prep, :313 run
probe.py	모드 분기 / 회귀 진입	set_nystrom_params:299 prepare_nystrom_regression:421 kernel_regression:490
entks/ntk_regression.py	Nystrom feature & ridge	_precompute:445 forward:301
dvutils/Data_Shapley.py	TMC 루프	run:338 tmc_one_iteration_per_point:205
task_data_selection.py	랭킹 선택 평가	sorted:211, dual eval:280/317